

Reviewer Pack — Lorentzian Hessian Gap of Spanning-Tree Polynomial Bounds Ma...

Entry 45433c968a7a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 10:48:46 UTC

Lorentzian Hessian Gap of Spanning-Tree Polynomial Bounds Max-CUT SOS Slack

Entry ID: 45433c968a7a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 10:48:46 UTC

1. Conjecture

Field A (mathematical branch): Lorentzian polynomials and combinatorial Hodge theory of matroids (Brändén-Huh 2020; Adiprasito-Huh-Katz), interpreted via Burton-Pemantle transfer currents on uniform random spanning trees **Field B** (complexity object): Sum-of-squares degree-2 (Goemans-Williamson / Delorme-Poljak) integrality slack for Max-CUT

Statement:

For a connected graph $G=(V,E)$ with $|V|=n$, $|E|=m$, let $f_G(x_e)=\sum_{T \text{ spanning tree}} \prod_{e \in T} x_e$ be its multilinear spanning-tree polynomial; f_G is Lorentzian, so its Hessian $H_G \in \mathbb{R}^{m \times m}$ at $x=1$, whose entries are $H_G[e,e'] = \tau\{e,e'\}$ for $e \neq e'$ and 0 on the diagonal, has signature $(+, -, -, \dots, -)$ with a unique positive eigenvalue $\lambda_1(H_G) > 0$ and remaining eigenvalues $\lambda_2 \geq \dots \geq \lambda_m$ all ≤ 0 . Define the Lorentzian Hessian gap $g(G) := \lambda_1(H_G) - \lambda_2(H_G) \geq \lambda_1(H_G) > 0$ (normalized by $\tau(G)$ so that off-diagonal entries equal $P(e,e' \in T)$). CONJECTURE: there exists an absolute constant $c > 0$ such that for every connected G with $n \geq 6$ and $m \geq n$, the Delorme-Poljak SOS-2 upper bound $DP(G) := m/2 + (n/4)|\lambda_{\min}(A_G)|$ exceeds the true MaxCut by at least $DP(G) - \text{MaxCut}(G) \geq c \cdot g(G) \cdot n$; a single graph G violating this for the empirically calibrated c falsifies the conjecture.

Rationale (proposer's reasoning):

The Lorentzian Hessian encodes the global negative-correlation structure of edges in a uniform random spanning tree (Burton-Pemantle), an instance-level matroid-Hodge invariant orthogonal to the cut-spectral structure that SOS-2 exploits; a large gap $g(G)$ signals matroid-balanced 'spread' that we predict the eigenvalue/SDP relaxation cannot reproduce, forcing slack against true MaxCut. The invariant is real-geometric (Hessian signatures over $\mathbb{R}$), not ring-extendable over a polynomial extension of $\mathbb{F}_2$, so the construction does not algebrize; and the property is read off the graph G as an instance, not from a Boolean truth table, so the natural-proofs largeness/usefulness combination does not apply. Combinatorial Hodge $\times$ SOS Max-CUT is essentially untouched (<5 papers).

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGES (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c565bccf876456c4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds $\times$ 5 sizes ($n \in \{8, 10, 12, 14, 16\}$) $\times$ 3 densities ($p \in \{0.4, 0.5, 0.6\}$) = 450 connected $G(n, p)$ instances, compute ratio $r(G) = (DP(G) - \text{MaxCut}(G)) / (g(G) \cdot n)$ for instances with $g(G) > 1e-6$. SUPPORTED iff min ratio $\geq 1e-3$ AND median ratio shows no monotone decay (median_r at $n=16 \geq 0.5 \cdot$ median_r at $n=8$). FALSIFIED iff any instance yields $r < 1e-3$ with $g > 1e-6$, or median_r(n) decays by $> 2\times$ from $n=8$ to $n=16$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Lorentzian polynomials spanning tree matroid Hessian eigenvalue - Delorme Poljak Max-Cut SOS integrality gap eigenvalue bound - transfer current spanning tree Brändén Huh Max-Cut semidefinite relaxation

Top relevant hits considered: - [<http://arxiv.org/abs/2206.02759v4>] Polynomials with Lorentzian Signature, and Computing Permanents via Hyperbolic Programming - [<http://arxiv.org/abs/2304.08399v3>] Dually Lorentzian Polynomials - [<http://arxiv.org/abs/2204.02097v2>] Simulated Annealing is a Polynomial-Time Approximation Scheme for the Minimum Spanning Tree Problem - [<http://arxiv.org/abs/1107.1630v3>] On the Integrality Gap of the Subtour LP for the 1,2-TSP - [<http://arxiv.org/abs/1207.5696v1>] Beyond Max-Cut: λ -Extendible Properties Parameterized Above the Poljak-Turzik Bound - [<http://arxiv.org/abs/2404.18142v1>] Revisiting Majumdar-Ghosh spin chain model and Max-cut problem using variational quantum algorithms - [<http://arxiv.org/abs/2601.02547v2>] Tree metrics and log-concavity for matroids - [<http://arxiv.org/abs/1812.10770v1>] Complex Semidefinite Programming and Max-k-Cut

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import combinations

# Helper functions for matrix operations
```

```

def identity_matrix(n):
    return [[1 if i == j else 0 for j in range(n)] for i in range(n)]

def transpose(matrix):
    return [list(row) for row in zip(*matrix)]

def matrix_add(A, B):
    return [[A[i][j] + B[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def matrix_subtract(A, B):
    return [[A[i][j] - B[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def matrix_multiply(A, B):
    n = len(A)
    m = len(B[0])
    result = [[sum(A[i][k] * B[k][j] for k in range(n)) for j in range(m)] for i in range(n)]
    return result

def determinant(matrix):
    if len(matrix) == 1:
        return matrix[0][0]
    det = 0
    for c in range(len(matrix)):
        submatrix = [row[:c] + row[c+1:] for row in matrix[1:]]
        sign = (-1) ** (c % 2)
        sub_det = determinant(submatrix)
        det += sign * matrix[0][c] * sub_det
    return det

def pinv(matrix):
    n = len(matrix)
    I = identity_matrix(n)
    augmented = [row + col for row, col in zip(matrix, I)]
    for i in range(n):
        pivot = augmented[i][i]
        if pivot == 0:
            raise ValueError("Matrix is not invertible")
        for j in range(n * 2):
            augmented[i][j] /= pivot
        for k in range(n):
            if k != i:
                factor = augmented[k][i]
                for j in range(n * 2):
                    augmented[k][j] -= factor * augmented[i][j]
    inverse = [row[n:] for row in augmented]
    return inverse

def max_cut(graph, n):
    max_cut_value = -1
    for subset in combinations(range(n), n // 2):
        cut_value = sum(graph[u][v] for u in subset for v in range(u + 1, n) if v not in subset)
        if cut_value > max_cut_value:
            max_cut_value = cut_value
    return max_cut_value

def delorme_poljak_bound(n, lambda_min):

```

```

    return n / 4 * abs(lambda_min)

def lorentzian_hessian_gap(H):
    eigenvalues = sorted(eigenvector_decomposition(H), reverse=True)
    return eigenvalues[0] - eigenvalues[1]

def eigenvector_decomposition(matrix):
    n = len(matrix)
    A = matrix_add(matrix, identity_matrix(n))
    for i in range(n):
        A[i][i] -= 2
    det_A = determinant(A)
    if det_A == 0:
        raise ValueError("Matrix is not invertible")
    eigenvalues = [1 + (det_A / (-n * n)) * ((-1) ** i) for i in range(n)]
    return eigenvalues

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [8, 10, 12, 14, 16]
    p_values = [0.4, 0.5, 0.6]
    c = 0.01
    results = []

    for n in n_values:
        for p in p_values:
            G = [[random.random() < p for _ in range(n)] for _ in range(n)]
            if any(sum(row) == 0 for row in G):
                continue

            # Compute Laplacian L and its inverse
            D = [sum(G[i]) for i in range(n)]
            L = [[D[i] - G[i][j] if i == j else -G[i][j] for j in range(n)] for i in range(n)]
            L_inv = pinv(L)

            # Compute Hessian entries
            p = [L_inv[i][i] + L_inv[j][j] - 2 * L_inv[i][j] for i, j in combinations(range(n), 2)]
            y = [[L_inv[u][u'] - L_inv[u][v'] - L_inv[v][u'] + L_inv[v][v'] for u', v' in combinations(range(n), 2)]
            H = [[p[i] * p[j] - y[i][j]**2 if i != j else 0 for j in range(len(p))] for i in range(len(p))]

            # Compute eigenvalues and gap
            g_G = lorentzian_hessian_gap(H)
            lambda_min_A_G = min(sum(G[u][v] for v in range(u + 1, n)) for u in range(n))
            DP_G = delorme_poljak_bound(n, lambda_min_A_G)

            # Compute MaxCut
            max_cut_value = max_cut(G, n)

            # Check ratio and record result
            if g_G > 1e-6:
                ratio = (DP_G - max_cut_value) / (g_G * n)
                results.append((n, p, DP_G, max_cut_value, g_G, ratio))

    if not results:
        return {
            "metric_name": "ratio",

```

```

        "metric_value": 0,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "no_valid_instances"
    }

min_ratio = min(ratio for _, _, _, _, _, ratio in results)
median_ratio = sorted(ratio for _, _, _, _, _, ratio in results)[len(results) // 2]
max_ratio = max(ratio for _, _, _, _, _, ratio in results)

return {
    "metric_name": "ratio",
    "metric_value": (min_ratio + median_ratio + max_ratio) / 3,
    "instances_tested": len(results),
    "conjecture_holds": min_ratio >= c and all(ratio >= c for _, _, _, _, _, ratio in results),
    "counterexample": "" if min_ratio >= c else f"ratio={min_ratio} < {c}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    ratios = [result["metric_value"] for result in results if result["instances_tested"] > 0]
    support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

    if all(r >= 1e-3 for r in ratios):
        print(f"RESULT: SUPPORTED mean={sum(ratios)/len(ratios)} std={math.sqrt(sum((r - (sum(ratios)/len(ratios))**2)/len(ratios)))}")
    elif any(r < 1e-3 and g > 1e-6 for _, _, _, _, g, r in results):
        print(f"RESULT: FALSIFIED counterexample=\"ratio<{min(r for _, _, _, _, _, r in results if r < 1e-3)}\"")
    else:
        print(f"RESULT: INCONCLUSIVE reason=median_ratio_decays_by_2x n_tested={len(ratios)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_66e7e22f.py", line 126, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_66e7e22f.py", line 81, in run_trial
    L_inv = pinv(L)
            ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_66e7e22f.py", line 44, in pinv
    inverse = [[col[i] for col in reduced_matrix] for i in range(n, 2*n)]
               ~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an `IndexError` in the custom `pinv()` routine before any trials completed, so no ratios were computed and the pre-registered support criterion cannot be evaluated. | next: Replace the hand-rolled `pinv` with `numpy.linalg.pinv` (or fix the row-reduction bounds) and rerun the full 450-instance sweep.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	486018
2	propose	claude_max	opus	0	0	307562
3	preregistration	claude_max	opus	0	0	7181
4	novelty	claude_max	opus	0	0	4696
5	novelty	claude_max	opus	0	0	8682
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22275
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18110
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18560
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21330
10	judge	claude_max	opus	0	0	5109

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 899523 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/45433c968a7a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/45433c968a7a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/45433c968a7a.tar.gz` (if generated)